ODOO 19 · HUMAN RESOURCES

ZKTeco ADMS Attendance

Biometric Device Integration for Odoo HR Attendance

Cloud-Push (ADMS / iClock) Protocol — SpeedFace-V5L

Technical Reference & Administrator Guide

Module: **zkteco_adms_attendance** · Version 19.0.1.0.0 · License LGPL-3

Verified: installs clean on a fresh database · 16/16 unit tests passing

Table of Contents

1. Overview & Architecture
2. Installation & Requirements
3. Data Model — Devices, Commands & Raw Logs
4. Attendance Policy Engine
5. ADMS / iClock Protocol Reference
6. Backend Usage Guide
7. Security & Access Rights
8. Device-Side Setup (SpeedFace-V5L) & Cron
9. Testing & Acceptance Criteria

1. Overview & Architecture

The **ZKTeco ADMS Attendance (Push)** module (`zkteco_adms_attendance`, version `19.0.1.0.0`) integrates ZKTeco biometric terminals — such as the SpeedFace-V5L — with Odoo's native `hr_attendance` application using the vendor's **ADMS / iClock cloud-push protocol**. It is published by AGM Global under the LGPL-3 license, is flagged as an installable Odoo *application*, and declares dependencies on `hr_attendance`, `resource`, and `mail`.

Functionally, the module receives raw punch events straight from the device, stores each one in an immutable audit log, and converts those logs into standard `hr.attendance` check-in / check-out records through a configurable **Attendance Policy Engine**. It adds a device-registration workflow, a live device dashboard, a stale-device offline cron, a safe reprocessing wizard, multi-company record rules, and dedicated access groups.

1.1 The Push Model

Unlike a traditional pull integration, Odoo **never initiates a connection to the terminal and never polls it**. The roles are inverted:

- **The device is the HTTP client.** The terminal's firmware periodically dials out to a set of well-known `/iclock/*` URLs on the Odoo server to fetch its configuration, upload transactions, and pull queued commands.
- **Odoo is the HTTP server.** The module exposes those routes through the `ZKTecoADMSController` class in `controllers/main.py`.

Because the terminal carries no Odoo session, every route is declared `auth='none'` with `csrf=False` and `save_session=False`, and every database access is performed through `sudo()`. Every reply is emitted as `text/plain` — the only content type the firmware understands. Devices are authenticated instead by a shared secret (`auth_token`), and unknown or still-`draft` terminals are rejected outright.

> **Exposed routes** (all `type='http'`, `auth='none'`):
>
> - `GET/POST /iclock/cdata` — handshake (GET) and data upload (POST).
> - `GET /iclock/getrequest` — device polls for queued commands.
> - `POST /iclock/devicecmd` — device acknowledges a command it executed.
> - `GET /iclock/ping` — lightweight health-check, always returns `OK`.

1.2 Handshake & Data-Push Flow

Two things happen in sequence: the terminal first performs a configuration **handshake**, then it **pushes** attendance transactions as employees punch. Both use the same `/iclock/cdata` endpoint — a `GET` for the handshake, a `POST` for the upload.

Sequence diagram (handshake + push)

1. **Handshake request.** On boot / at intervals the device issues `GET /iclock/cdata?SN=<serial>`, optionally presenting its token.
2. **Device lookup & auth.** The controller resolves the serial via `_get_registered_device(sn)`. If the device is missing or in `draft` state it returns `ACCESS DENIED` (HTTP 401). If a token is configured it must match the `Authorization` header (optionally `Bearer <token>`), the `X-ZKTeco-Token` header, or a `token` parameter; otherwise the request is denied and logged with the serial and remote address. A device with no token set is accepted (open enrollment).
3. **Presence update.** The controller calls `mark_online()`, refreshing `last_seen` and moving the device to the `online` state.
4. **Handshake response.** Odoo returns the option block built by `_build_handshake_options()`, starting with `GET OPTION FROM: <SN>` followed by the device's operating parameters (see payload below).
5. **Employee punches.** A worker presents a face / finger / card at the terminal, which records the transaction locally.
6. **Data push.** The device uploads via `POST /iclock/cdata?SN=<serial>&table=ATTLOG` with the raw punch lines in the request body.
7. **Ingest & store.** The controller routes `ATTLOG` bodies to `ingest_attlog()`, which parses each line, converts the device-local timestamp to naive UTC, maps status / verify codes, de-duplicates offline replays, and creates immutable `zkteco.biometric.attendance.log` records.
8. **Match & apply policy.** For the freshly created logs the engine runs `_match_employee()` then `_process_logs()`, applying the assigned Attendance Policy to create or extend the corresponding `hr.attendance` records.
9. **Acknowledge.** Odoo replies `OK: <count>` where *count* is the number of newly stored logs, telling the firmware the batch was accepted so it will not resend it.

The handshake body returned in step 4 looks like this:

```
GET OPTION FROM: <serial_number>
Stamp=9999
OpStamp=9999
ErrorDelay=30
Delay=<heartbeat_interval, default 30>
TransTimes=00:00;14:05
TransInterval=<transaction_interval, default 1>
TransFlag=1111000000
TimeZone=<device tz offset in minutes>
Realtime=1
Encrypt=0
```

> On a `POST /iclock/cdata` the `table` parameter selects the handler (it defaults to `ATTLOG`): an `ATTLOG` body is ingested as punches and an `OPERLOG` body is passed to `ingest_operlog()` — both acknowledged with `OK: <count>`. Any other table (e.g. `OPTIONS` / `BIODATA`) is simply acknowledged with a bare `OK`.

> Punch timestamps arrive in the device's local time. Ingestion converts them to naive UTC using the device's configured timezone (`tz`). A device with an incorrect timezone will therefore store attendance at the wrong absolute time — set the device timezone correctly before confirming it.

1.3 Feature List

- **ADMS / iClock HTTP controller** — implements the handshake, ATTLOG / OPERLOG push, and the command queue over the standard `/iclock/*` routes.
- **Device registration workflow** — terminals move through `Draft → Confirmed → Online / Offline`; confirming a device auto-generates its `auth_token` when none is set.
- **Immutable raw punch log** — every transaction is preserved as an audit record, with batch-level and database-level de-duplication of offline replays.
- **Attendance Policy Engine** — four selectable modes (*Device Status Code*, *First Check Time*, *Flexible Period*, *Work Schedule / Calendar*) with tunable dedup, minimum-gap, and shift-span windows.
- **Safe reprocessing wizard** — re-maps existing logs to `hr.attendance` without destroying the raw audit trail.
- **Live device dashboard & offline cron** — a scheduled job marks stale devices offline and can raise admin activities.
- **Command queue** — pending commands are handed to the device on its `getrequest` poll and acknowledged back through `devicecmd`.
- **Multi-company & security** — dedicated record rules and access groups scope data per company.

1.4 Component Map

The module is organised into a thin protocol controller backed by a set of persistent models, a policy engine, a wizard, a cron job, and a security layer.

Layer	Artifact	Responsibility
Controller	`controllers/main.py` — `ZKTecoADMSController`	Terminates the ADMS / iClock HTTP protocol: handshake, ATTLOG / OPERLOG ingest, command delivery and acknowledgement, ping.
Device model	`zkteco.biometric.device`	Registered terminal: serial, token, timezone, intervals, presence state; builds the handshake and command payloads.
Raw log model	`zkteco.biometric.attendance.log`	Immutable punch audit records; ingest, employee matching, and policy processing entry points.
Command model	`zkteco.device.command`	Outbound command queue delivered on the device poll and marked done / error on acknowledgement.
Policy engine	`hr.attendance.policy` + `zkteco.employee.policy.line`	Converts raw punches into `hr.attendance` records according to the selected `mode`.
HR extensions	`hr.employee`, `hr.attendance`	Link employees to biometric IDs / policies and annotate attendances with their source.

Layer	Artifact	Responsibility
Configuration	`res.company`, `res.config.settings`	Company-level and global settings for the integration.
Wizard	`attendance.reprocess.wizard`	Re-runs the policy engine over selected logs without data loss.
Cron	`data/cron.xml` → `_cron_mark_offline()`	Flags devices offline when they stop checking in.
Security	`security/security.xml`, `security/ir.model.access.csv`	Access groups, model ACLs, and multi-company record rules.

> Because the device drives every exchange, integration health is best judged from the Odoo side: a terminal that recently hit any `/iclock/*` route will show a fresh `last_seen` and an `online` state, while the offline cron catches devices that have gone silent.

2. Installation & Requirements

This chapter lists the platform and module prerequisites for **ZKTeco ADMS Attendance (Push)** and walks through a clean installation. All facts below are taken directly from the module's `__manifest__.py`.

2.1 Platform Requirements

Requirement	Value
Odoo version	19.0 (manifest `version` is `19.0.1.0.0`)
Edition	Community *or* Enterprise — no Enterprise-only modules are required
Technical name	`zkteco_adms_attendance`
License	LGPL-3
Category	Human Resources / Attendances
Marked as	`application` = True, `installable` = True, `auto_install` = False

> **Community is fully supported.** The module depends only on `hr_attendance`, `resource`, and `mail` — all of which ship with Odoo Community. You do *not* need Odoo Enterprise or any paid app to run this integration.

2.2 Module Dependencies

The manifest's `depends` list declares three modules. Odoo installs them automatically before this module if they are not already present:

Dependency	Why it is needed
`hr_attendance`	Provides the `hr.attendance` and `hr.employee` models. The Policy Engine converts raw device punches into `hr.attendance` check-in / check-out records, and the module extends the HR Attendances application with its own menus.
`resource`	Supplies working-schedule / calendar data (`resource.calendar`) used by the Calendar-based attendance policy to interpret punches against an employee's working hours.
`mail`	Enables chatter, tracking, and scheduled activities. The module posts admin activities (for example when a device goes stale/offline) and uses message tracking on device and log records.

2.3 Making the Module Discoverable

The module lives in the repository under `custom_addons/zkteco_adms_attendance`. Odoo can only see it if that `custom_addons` folder is on the server's `addons_path`.

- **Config file:** add the directory to the `addons_path` line in `odoo.conf` (comma-separated), for example:

```
[options]
addons_path = /odoo/addons,/odoo/custom_addons
```

- **Or on the command line:** pass `--addons-path` when launching the server.

```
odoo-bin --addons-path=/odoo/addons,/odoo/custom_addons
```

> Point `addons_path` at the folder that *contains* `zkteco_adms_attendance` (i.e. `custom_addons`), **not** at the module folder itself. If the path is wrong the module will not appear in the Apps list even after updating.

2.4 Installation Procedure

Follow these steps for a first-time install:

1. Copy or clone the `zkteco_adms_attendance` folder into your `custom_addons` directory.
2. Ensure `custom_addons` is on the `addons_path` (see section 2.3) and restart the Odoo service so the new path is loaded.

3. Log in as an Administrator and open `Apps`.
4. Click `Update Apps List` (you may need Developer Mode enabled) so Odoo re-scans the addons paths.
5. Remove the default *Apps* filter, search for **ZKTeco**, locate *ZKTeco ADMS Attendance (Push)*, and click `Install`.
6. Odoo automatically pulls in the `hr_attendance`, `resource`, and `mail` dependencies and loads the module's security rules, cron jobs, default policy data, views, and menus.

2.4.1 Installing from the Command Line

To install (or update) without the UI, use the module's technical name with the `-i` (install) or `-u` (upgrade) flag against your database:

```
# Fresh install
odoo-bin -c odoo.conf -d your_db -i zkteco_adms_attendance

# Upgrade an already-installed copy
odoo-bin -c odoo.conf -d your_db -u zkteco_adms_attendance
```

If `addons_path` is not set in `odoo.conf`, add `--addons-path=/odoo/addons,/odoo/custom_addons` to the same command.

2.5 First-Run Expectations

After a successful install you should see the following, confirming the module loaded correctly:

- The module appears as **Installed** in `Apps` (it registers as an application, so it may also surface as a top-level app tile).
- New configuration menus appear inside the HR Attendances application under `Attendances > Configuration` — this is where biometric devices, attendance policies, and related settings are managed (menus are defined in `views/zkteco_menus.xml`).
- Default attendance-policy records and the stale-device offline cron job are created from the module's data files (`data/attendance_policy_data.xml` and `data/cron.xml`).
- Dedicated access groups and multi-company record rules are installed (from `security/security.xml` and `security/ir.model.access.csv`), so grant users the appropriate group to expose the menus to non-admin staff.

> If the `Attendances > Configuration` menus do not show up, confirm you are looking inside the *Attendances* app, that your user has the module's access group, and that the install finished without errors in the server log. A quick `-u zkteco_adms_attendance` re-run reloads the menu and view definitions.

3. Data Model — Devices, Commands & Raw Logs

This module persists three related records: the **device** (one row per physical ZKTeco terminal), the **command queue** (outbound instructions the device collects when it polls), and the **raw attendance log** (one row per punch pushed by the device). All three live under `models/` and are wired together by the ADMS push controller. This section documents the technical model names, every persisted field, the key methods, the device state machine and the raw-log de-duplication scheme.

> Timestamps sent by the device are in *device-local* time. The ingest path converts them to **naive UTC** before storing them in `punch_datetime`, using the device's own `tz` field. Keep every device's timezone correct or punches will land on the wrong calendar day.

3.1 Device — `zkteco.biometric.device`

Purpose. Represents a single registered ZKTeco terminal (defined in `models/biometric_device.py`). It stores the identity and network configuration of the device, the shared `auth_token` validated on every push, the presence state machine, and the timing knobs (upload interval / heartbeat) that are pushed to the device in the ADMS handshake. It inherits `mail.thread` and `mail.activity.mixin` for chatter and activities, and is ordered by `device_name`. Serial number is enforced unique by the `_serial_number_uniq` constraint (`unique(serial_number)`).

Field	Type	Description
device_name	Char (required, tracked)	Human-readable name of the terminal.
serial_number	Char (required, indexed, copy=False, tracked)	Device serial reported in every ADMS request; the primary lookup key and unique across the database.
mac_address	Char	Device MAC address (informational).
ip_address	Char	Device IP address (label "IP Address").
subnet_mask	Char	Network subnet mask (informational).
gateway	Char	Network gateway (informational).
port	Integer (default 8069)	Server port the device connects to.
location	Char	Free-text physical location.
tz	Selection (all pytz zones)	Device timezone; defaults to the current user's tz or `UTC`. Drives local→UTC conversion of punches and the handshake TimeZone offset.
firmware_version	Char (readonly, copy=False)	Firmware string; auto-filled by `mark_online` when the device reports it.
verify_mode_palm	Boolean (default False)	Whether palm verification is enabled ("Palm").
verify_mode_face	Boolean (default True)	Whether face verification is enabled ("Face").
verify_mode_fingerprint	Boolean (default True)	Whether fingerprint verification is enabled ("Fingerprint").
verify_modes_display	Char (computed)	Comma-joined list of the enabled modes (Face, Fingerprint, Palm).
state	Selection: draft / confirmed / online / offline (default draft, required, tracked, copy=False)	Presence/workflow state machine (see 3.4).
last_seen	Datetime (readonly, copy=False, tracked)	UTC timestamp of the last contact; refreshed by `mark_online` and evaluated by the offline cron.
auth_token	Char (copy=False)	Shared secret validated on every push; auto-generated (uuid4 hex) on confirm if empty.
transaction_interval	Integer (default 1)	Upload interval in minutes ("Upload Interval (min)"); sent as `TransInterval`.
heartbeat_interval	Integer (default 30)	Heartbeat in seconds ("Heartbeat (sec)"); sent as `Delay`.

Field	Type	Description
company_id	Many2one res.company (required, indexed)	Owning company; defaults to the current company.
command_ids	One2many → zkteco.device.command	Commands queued for this device.
log_ids	One2many → zkteco.biometric.attendance.log	Raw punches received from this device.
log_count	Integer (computed)	Number of related logs (via read_group).
command_count	Integer (computed)	Total number of related commands.
pending_command_count	Integer (computed)	Count of commands still in `pending` state.

Key device methods

- **`_get_registered_device(serial_number)`** — model method that resolves a device by serial number (`sudo().search([('serial_number','=',…)], limit=1)`), returning an empty recordset if no serial is supplied. This is the entry lookup used by the ADMS controller.
- **`_get_timezone_offset_minutes()`** — returns the device timezone's UTC offset in whole minutes (0 on missing/invalid tz), used to build the handshake `TimeZone` line.
- **`action_confirm()`** — moves a `draft` device to `confirmed` and, if `auth_token` is empty, generates one (`uuid.uuid4().hex`).
- **`action_reset_to_draft()`** / **`action_set_offline()`** — write `state = 'draft'` / `'offline'` respectively.
- **`action_upload_settings()`** — refuses on a draft device; otherwise queues an `INFO` command and a `SET OPTION Realtime=1,TransInterval=…,Delay=…` command (built from the interval/heartbeat fields) and posts a chatter note.
- **`mark_online(firmware_version=None)`** — records a contact: refreshes `last_seen`, stores a changed firmware string, and promotes the device to `online` when it is currently `confirmed`, `offline` or `online` (a `draft` device is never auto-promoted).
- **`_build_handshake_options()`** — returns the ADMS `GET /iclock/cdata` handshake body (CRLF-joined) that tells the device how to behave. See payload below.
- **`queue_command(command_type, command_text)`** — creates and returns a new `pending` command on `zkteco.device.command` for this device.
- **`get_pending_commands_payload()`** — for `GET /iclock/getrequest`: searches this device's `pending` commands ordered oldest-first, renders each as `C:<id>:<command_text>` on its own line, calls `mark_sent()` on them, and returns the joined lines — or the literal string `OK` when the queue is empty.
- **`_cron_mark_offline()`** — scheduled job (see 3.4) that flips stale online devices to `offline` and schedules a to-do activity for the responsible user.

The handshake body returned by `_build_handshake_options()` looks like:

```
GET OPTION FROM: <serial_number>
Stamp=9999
OpStamp=9999
ErrorDelay=30
Delay=<heartbeat_interval>
TransTimes=00:00;14:05
TransInterval=<transaction_interval>
TransFlag=1111000000
TimeZone=<offset_minutes>
Realtime=1
Encrypt=0
```

3.2 Command — `zkteco.device.command`

Purpose. Outbound command queue (defined in `models/zkteco_device_command.py`). Rows are created in `pending` state (typically by `device.queue_command(...)`), collected by the device on its next `getrequest` poll (flipping them to `sent`), and finally acknowledged by the device (moving them to `done` or `error`). The model is ordered `create_date desc`, so the newest command shows first in the UI while the payload builder explicitly re-sorts oldest-first for delivery.

Field	Type	Description

Field	Type	Description
device_id	Many2one → zkteco.biometric.device (required, ondelete=cascade, indexed)	Target device; commands are deleted with the device.
company_id	Many2one res.company (related, stored, indexed)	Mirrors the device's company for record rules/filtering.
command_type	Selection (required, default custom)	Logical category: `info` (Query Info), `reboot`, `sync_time`, `sync_users`, `clear_data` (Clear Attendance Data), `unlock` (Unlock Door), `option` (Set Options), `custom`.
command_text	Char (required)	Raw ADMS command body actually sent to the device, e.g. `REBOOT` or `CLEAR LOG`.
state	Selection: pending / sent / done / error (default pending, required, indexed)	Delivery/acknowledgement lifecycle.
return_code	Char (copy=False)	Return code reported by the device on acknowledgement.
response	Char (copy=False)	Optional response text from the device.
sent_date	Datetime (copy=False)	When the command was handed to the device.
ack_date	Datetime (copy=False)	When the device acknowledged execution.

Key command methods

- `mark_sent()` — sets `state = 'sent'` and stamps `sent_date` with now. Called in bulk by `get_pending_commands_payload()` once the lines are handed to the device.
- `mark_done(return_code=None, response=None)` — acknowledges execution. It treats any return code *not* in `('0', 0, None, '')` as a failure: on a success code the command becomes `done`, otherwise `error`. It stamps `ack_date` and stores the stringified return code / response when provided.

> Delivery format: each pending command is serialized as `C:<id>:<command_text>`. The numeric `<id>` is the record's database id, which the device echoes back on acknowledgement so the controller can locate and `mark_done()` the exact command.

3.3 Raw Attendance Log — `zkteco.biometric.attendance.log`

Purpose. One row per raw punch pushed by a device (defined in `models/biometric_attendance_log.py`). It is the immutable landing table for ADMS `ATTLOG` data: it keeps the original line, the normalized UTC timestamp, the mapped punch/verify types, the matched employee and the processing verdict. Downstream, the processing engine turns matched punches into `hr.attendance` records. The model is ordered `punch_datetime desc, id desc`.

Field	Type	Description
device_id	Many2one → zkteco.biometric.device (required, ondelete=cascade, indexed)	Source device.
company_id	Many2one res.company (related, stored, indexed)	Company of the source device.
biometric_user_id	Char (required, indexed)	"Biometric ID" — the enrollment PIN reported by the device; matched against `hr.employee.biometric_user_id`.
employee_id	Many2one → hr.employee (indexed, ondelete=set null)	Employee resolved from the biometric id; empty until matched.
punch_datetime	Datetime (required, indexed)	"Punch Time" stored in **naive UTC** (converted from device-local time).
punch_type	Selection (default unknown)	Mapped from the status code: `in`, `out`, `break_out`, `break_in`, `ot_in`, `ot_out`, `unknown`.
status_code	Char	Raw device status field taken verbatim from the ATTLOG line.

Field	Type	Description
verify_mode	Selection (default other)	Mapped from the verify code: `password`, `fingerprint`, `card`, `face`, `palm`, `other`.
raw_payload	Char	The original raw ATTLOG line as received (audit trail).
processed	Boolean (default False, indexed)	Whether the engine has finished with this punch.
process_state	Selection (default pending, indexed)	Verdict: `pending`, `done` (Processed), `unmatched` (Unmatched Employee), `duplicate`, `skipped`, `error`.
process_note	Char	Human-readable detail for the current process state (e.g. the error text or unmatched reason).
hr_attendance_id	Many2one → hr.attendance (ondelete=set null, copy=False, indexed)	Attendance record generated from this punch, for traceability.

Code maps

Two module-level dictionaries translate the numeric device codes (also exposed as class attributes `STATUS_MAP` / `VERIFY_MAP`):

STATUS_MAP (status → punch_type)	VERIFY_MAP (verify → verify_mode)
0 → in	0 → password
1 → out	1 → fingerprint
2 → break_out	2 → card
3 → break_in	15 → face
4 → ot_in	9 → palm
5 → ot_out	(fallback) → other
(fallback) → unknown	

Key log methods

- `_parse_attlog_line(line)` — parses one raw ATTLOG line. It splits on tabs (whitespace tolerated); with no tabs it falls back to whitespace tokens and reconstructs the two-token `date time` back into one datetime. It validates the timestamp against `ATTLOG_DT_FORMAT` (`%Y-%m-%d %H:%M:%S`) and returns a dict with `biometric_user_id`, `datetime_str`, `status_code`, `verify_code` and `raw` — or `None` for blank/malformed lines.
- `_local_str_to_utc(datetime_str, tz)` — localizes the device-local string into the device timezone and returns a naive UTC `datetime` (handles non-existent/ambiguous DST wall-clock times), or `None` on failure.
- `ingest_attlog(device, body_text)` — the main ingestion entry point called by the ADMS controller. See the de-duplication walkthrough in 3.5. Returns a tuple `(created_recordset, created_count)`.
- `ingest_operlog(device, body_text)` — minimal, never-raising handler that scans the operation log and returns the count of `USER` lines seen so the controller can acknowledge the upload.
- `_match_employee()` — resolves `employee_id` from the biometric id, preferring an employee in the punch's own company and falling back to any employee carrying that biometric id. Unmatched punches are flagged `process_state = 'unmatched'` with a note and left unprocessed; a later successful match clears a prior `unmatched` verdict back to `pending`.
- `_process_logs()` — the processing engine: skips punches with no employee (marks them `unmatched`), groups the remaining matched, unprocessed punches by employee, then by their *local* calendar date, resolves the applicable `hr.attendance.policy` per date, and calls `policy.apply_to_logs(...)` on the ascending-sorted punches. Policy failures flag that date's logs as `error` without stopping other employees/dates; punches linked to an attendance are finally normalized to `processed = True` / `done`.
- `action_reprocess()` — button that resets the selected punches (`processed=False`, `process_state='pending'`, clears note and `hr_attendance_id`) and re-runs `_match_employee()` then `_process_logs()`.

3.4 The device state machine (draft → confirmed → online → offline)

The `state` field combines a manual setup workflow with an automatic presence tracker:

- **draft** — freshly created and not yet activated. A draft device is rejected by `action_upload_settings()` and is never auto-promoted to online by `mark_online()`.

- **confirmed** — reached via `action_confirm()`, which also generates the `auth_token` if absent. The device is now allowed to be promoted on contact.
- **online** — set by `mark_online()` on any successful push contact, but only when the device is already `confirmed`, `offline` or `online`. Each contact also refreshes `last_seen`.
- **offline** — set either manually (`action_set_offline()`) or automatically by the `_cron_mark_offline()` job.

The offline cron scans devices currently in `online` state and compares `last_seen` against a per-company threshold (`company_id.zkteco_device_offline_minutes`, defaulting to 5 minutes). Devices with no `last_seen` or a stale one are flipped to `offline` in bulk, and a *To-Do* activity ("Biometric device offline") is scheduled for the responsible user — the device's creator if still active, otherwise an HR Attendance manager, otherwise the current user. `action_reset_to_draft()` can return any device to `draft`.

> Because `mark_online()` only promotes devices in `confirmed/offline/online`, a device that has gone `offline` automatically recovers to `online` the moment it next contacts the server — no manual intervention is needed.

3.5 Raw-log de-duplication

ZKTeco terminals buffer punches while the network is down and *replay* them once connectivity returns, so the same punch can arrive multiple times. The log model prevents duplicates with a database-level unique constraint declared through the Odoo 19 `models.Constraint` API (the classic `_sql_constraints` list is no longer honored):

```
_punch_uniq = models.Constraint(
    'unique(device_id, biometric_user_id, punch_datetime, status_code)',
    'This punch was already recorded (duplicate replay ignored).',
)
```

The uniqueness key is therefore the tuple **(device_id, biometric_user_id, punch_datetime, status_code)**. The attribute name `_punch_uniq` produces a DB constraint whose short name is `punch_uniq`. Two identical punches (same device, same PIN, same UTC instant, same status) can never both be stored.

`ingest_attlog()` defends against replays in three layers so a duplicate never aborts a whole batch:

1. **In-batch de-dup.** While parsing, each line's key is tracked in a `seen_keys` set; repeats within the same POST body are dropped before any DB work.
2. **Pre-search.** Existing rows matching the batch's PINs and timestamps are fetched in one query and their keys collected into `existing_keys`, so already-stored punches are skipped cheaply without hitting the constraint.
3. **Savepoint-guarded create.** Each surviving row is created inside its own `cr.savepoint()`. If a concurrent replay still slips through and violates `punch_uniq`, the resulting `IntegrityError` is caught, that one row is rolled back and skipped (logged at debug level), and the ingest continues with the rest of the batch.

Note the constraint keys on `status_code`, so an in/out pair at the exact same second (different status) is legitimately kept as two rows; only a byte-for-byte identical status replay is suppressed. Only the punches that were actually created are then handed to `_match_employee()` and `_process_logs()`.

> De-duplication hinges on a stable `punch_datetime`. Since that value is the device-local time converted through the device's `tz`, changing a device's timezone after punches exist shifts newly-computed timestamps and can let an otherwise-duplicate replay through under a different key. Set the timezone correctly before the device starts pushing and avoid changing it in production.

4. Attendance Policy Engine

The **Attendance Policy Engine** is the layer that converts a raw stream of biometric punches (stored as `zkteco.biometric.attendance.log` records) into paired `hr.attendance` check-in / check-out sessions. All of the pairing intelligence lives in the model `hr.attendance.policy` (file `models/hr_attendance_policy.py`). A punch never becomes an attendance directly; instead the log-processing engine resolves *which* policy governs a given employee on a given day, and that policy decides how the day's punches are folded into sessions.

> **Where processing is triggered.** `zkteco.biometric.attendance.log._process_logs()` (in `models/biometric_attendance_log.py`) groups every matched, unprocessed punch first by `employee_id`, then by the punch's *local calendar date* (using the employee timezone via `employee._get_tz()`). For each `(employee, local_date)` group it calls `Policy._resolve_for_employee(employee, local_date)` and then `policy.apply_to_logs(date_logs)`, iterating dates chronologically. Punches with no matched employee are flagged `process_state = unmatched` and skipped; any exception inside a policy is caught and the affected punches are marked `process_state = error` so other employees and dates keep flowing.

4.1 The `hr.attendance.policy` model

A policy is a small, reusable configuration record. Its `mode` selects one of four pairing algorithms; the remaining numeric fields tune de-duplication and session-boundary behaviour that *every* mode shares. The model is `_order = 'name'`.

Field	Type	Label / default	Purpose
`name`	Char	required	Human-readable policy name (e.g. "Standard Shift", "Flexible Office").
`mode`	Selection	required, default `device_status`	Selects the pairing algorithm. Values: `device_status` (Device Status Code), `first_check` (First Check Time), `flexible` (Flexible Period), `calendar` (Work Schedule (Calendar)).
`flexible_window_hours`	Float	"Max Shift Span (h)", default `12.0`	Used by the `flexible` mode: a punch further than this many hours from the open check-in is treated as a forgotten check-out and starts a new attendance.
`dedup_window_seconds`	Integer	"Duplicate Window (s)", default `60`	Any punch whose time lands within this many seconds of an existing check-in or check-out edge for the employee is ignored as a device double-read (marked `duplicate`). `0` disables de-duplication.
`min_gap_minutes`	Integer	"Min Gap After Check-out (min)", default `1`	A new check-in occurring within this many minutes of the previous check-out re-opens (merges back into) that just-closed session instead of fragmenting the work period. `0` disables merging.
`active`	Boolean	default `True`	Standard archive flag.
`company_id`	Many2one → res.company	default `env.company`	Owning company; used during resolution to prefer company-scoped policies.
`note`	Text	—	Free-form documentation of the policy's intent.

> Regardless of mode, every session the engine creates or closes is written with `in_mode = 'technical'` / `out_mode = 'technical'` on `hr.attendance`, and each processed punch is linked back to its resulting attendance via `hr_attendance_id` and stamped `processed = True`, `process_state = 'done'`.

4.2 The four pairing modes

`apply_to_logs()` defensively re-sorts the punches ascending by `(punch_datetime, id)` and dispatches to one of four private helpers based on `mode`. The examples below all assume the punch timestamps shown; times are illustrative local times.

4.2.1 Device Status Code (`device_status`)

Handler `_apply_device_status()`. This mode *trusts the status code the device reported* for each punch. The raw device status is mapped to a `punch_type` at ingestion time via `STATUS_MAP` (`0→in, 1→out, 2→break_out, 3→break_in, 4→ot_in, 5→ot_out`). The handler then buckets those types:

- **In types** — `in`, `break_in`, `ot_in`: open a new attendance (or, if one is already open, simply link the punch to it, treating it as a missed check-out).

- **Out types** — `out`, `break_out`, `ot_out`: close the currently open attendance; if none is open, one is created at that instant and immediately closed.
- Anything else (`unknown`) is finalized as `skipped`.

Worked example. Punches: `09:00 (status 0 → in)`, `18:00 (status 1 → out)`. The 09:00 punch opens an attendance with `check_in = 09:00`; the 18:00 punch closes it with `check_out = 18:00`. Result: one session 09:00 → 18:00.

> Because `break_out` / `break_in` and `ot_out` / `ot_in` fall into the out/in buckets, a device configured to send break punches will split a shift around the break. Use this mode only when the hardware's status codes reliably reflect true in/out intent.

4.2.2 First Check Time (`first_check`)

Handler `_apply_first_check()`. Status codes are ignored. The *first* punch of the local day opens the attendance; every later punch simply pushes the `check_out` forward to the latest punch time. A punch at or before the opening punch is just linked with no effect, and a single lone punch leaves the day open (no check-out). On reprocessing it reuses an attendance already recorded for that local day rather than duplicating it.

Worked example. Punches on one local day: `08:55, 12:30, 13:15, 17:40`. The 08:55 punch opens the session; 12:30 moves the check-out to 12:30, 13:15 to 13:15, 17:40 to 17:40. Result: a single session 08:55 → 17:40 (mid-day punches are absorbed).

4.2.3 Flexible Period (`flexible`)

Handler `_apply_flexible()`. A pure toggle bounded by `flexible_window_hours` (default 12h). With an open attendance:

- if the new punch is within `0 … window` hours of the open check-in, it **closes** the session;
- if it is beyond the window, the open session is assumed to be a forgotten check-out: it is closed at *its own check-in time* (a zero-length record) and a **new** attendance is opened at the new punch.

With no open attendance, the punch opens one (subject to the `min_gap_minutes` merge described below).

Worked example. With `flexible_window_hours = 12`: punch `09:00` opens a session; punch `17:00` (span 8h ≤ 12h) closes it → 09:00 → 17:00. Now suppose the next punch is `09:00 the next day` (span ≈ 24h > 12h) while a session is somehow still open: the stale session is closed at its own check-in and a fresh session opens at that next-day 09:00.

4.2.4 Work Schedule / Calendar (`calendar`)

Handler `_apply_calendar()`. Each punch is *classified* as a check-in or check-out by comparing it to the employee's actual working intervals. The engine reads the employee's `resource_calendar_id` and `resource_id` and computes work periods around the punch window via `_work_intervals_batch` (converted to naive UTC in `_calendar_work_periods`). For each punch, `_classify_by_calendar` finds the nearest boundary: closest to a *shift start* ⇒ `in`, closest to a *shift end* ⇒ `out`. If no boundary lies within 12 hours of the punch it returns `None` and the code falls back to a plain toggle (out if a session is open, else in). Classified punches are then paired with the same open/close toggle logic as the other modes.

Worked example. The employee's calendar defines a 09:00–17:00 shift. Punch `08:52` is nearest the 09:00 start ⇒ classified `in` ⇒ opens the session. Punch `17:05` is nearest the 17:00 end ⇒ classified `out` ⇒ closes it. Result: session 08:52 → 17:05, even though the device reported no status codes.

> **Graceful degradation.** If the employee has no calendar/resource, or no scheduled work exists around the punches, or interval computation raises, `calendar` mode silently falls back to `flexible` pairing rather than failing ingestion.

4.3 Shared safeguards: de-duplication and gap merging

Before any pairing, every mode calls `_is_duplicate_punch()`: if the punch time is within `dedup_window_seconds` of an existing check-in or check-out edge for that employee, the punch is finalized as `process_state = duplicate` and ignored. When a mode opens a check-in it routes through `_checkin_respecting_gap()`: if the previous attendance's check-out is within `min_gap_minutes` of the new punch, that session is re-opened (its check-out cleared) and the punch merged into it, avoiding fragmenting one work period because of a stray out/in pair.

4.4 Policy resolution priority (`_resolve_for_employee`)

`_resolve_for_employee(employee, punch_date)` is guaranteed to return a usable policy — it never returns an empty recordset. It walks a priority chain and stops at the first match:

1. **Employee policy line by date range.** Among the employee's `zkteco_policy_line_ids`, keep lines whose `date_from` / `date_to` bracket `punch_date` (open-ended bounds allowed) and that reference a policy. If several match, the one with the most recent `date_from` (most specific assignment) wins.

2. **Employee default policy.** The employee's `biometric_policy_id`, if set.
3. **Company default policy.** The company's `zkteco_default_policy_id`, if set.
4. **First `device_status` policy** for the company (or a company-agnostic one), then failing that, *any* policy for the company/global.
5. **Last-resort fallback.** A brand-new, in-memory (`new()`) policy named "Default (Device Status)" with `mode = device_status`. This guarantees punches are always paired even in a freshly installed database with no configured policies.

4.5 Per-employee assignment: `zkteco.employee.policy.line`

This model (`_order = 'date_from desc'`) lets you assign different policies to an employee over time — for example a fixed-shift policy this quarter and a flexible one next quarter.

Field	Type	Notes
`employee_id`	Many2one → hr.employee	required, `ondelete='cascade'`, indexed.
`policy_id`	Many2one → hr.attendance.policy	required, `ondelete='restrict'` (a policy in use cannot be deleted).
`date_from`	Date	Optional lower bound; empty means "since forever".
`date_to`	Date	Optional upper bound; empty means "until forever".
`company_id`	Many2one (related, stored)	Derived from `employee_id.company_id`.

A model constraint `_check_date_order` raises a `ValidationError` if `date_from` is after `date_to`.

4.6 `hr.employee` additions

The module extends the employee record with the biometric identity and its policy wiring (all fields are restricted to the `hr_attendance.group_hr_attendance_officer` group):

- `biometric_user_id` (Char, "Biometric ID", indexed, `copy=False`) — the enrollment ID / PIN on the device; this is the key used to match a punch to an employee.
- `biometric_policy_id` (Many2one → hr.attendance.policy, "Attendance Policy") — the employee's default policy (priority step 2 above).
- `zkteco_policy_line_ids` (One2many → zkteco.employee.policy.line) — the date-ranged assignments (priority step 1).
- `biometric_log_ids` (One2many → zkteco.biometric.attendance.log) — this employee's raw punches.
- `biometric_log_count` (Integer, computed via `_compute_biometric_log_count` using a batched `_read_group`) — drives a smart button.

The method `action_view_biometric_logs()` backs the smart button: it opens the `action_zkteco_biometric_attendance_log` window action filtered and defaulted to the current employee (with a safe inline fallback action if the XML id is missing).

4.7 `hr.attendance` additions

Attendance records carry back-references so a session can be traced to the punches that produced it:

- `biometric_log_ids` (One2many → zkteco.biometric.attendance.log via `hr_attendance_id`) — the punches linked to this attendance.
- `is_biometric` (Boolean, computed & stored) — `True` when at least one biometric log is linked.
- `biometric_device_id` (Many2one → zkteco.biometric.device, computed & stored) — the device of the first linked log.

Both computed fields are produced together by `_compute_is_biometric`, which depends on `biometric_log_ids`; the device is taken from the first linked log (`logs[:1].device_id`).

4.8 Company & settings fields

`res.company` is extended (and surfaced on the Settings screen via `res.config.settings` as `related` read/write fields):

- `zkteco_default_policy_id` (Many2one → hr.attendance.policy, "Default Attendance Policy") — the company-wide fallback used when an employee has no specific assignment (priority step 3).
- `zkteco_device_offline_minutes` (Integer, "Device Offline After (min)", default `5`) — a device that has not reported within this many minutes is marked offline by the scheduled action (see the device-monitoring section).

> To roll out biometric attendance quickly: create one `hr.attendance.policy`, set it as the company's `zkteco_default_policy_id` under `Settings`, and only add per-employee `Policy Assignments` for the exceptions. Because resolution always falls back to a device-status policy, punches are still paired even before you configure anything.

5. ADMS / iClock Protocol Reference

All endpoints in this section are implemented by `ZKTecoADMSController` in `controllers/main.py`. In the ADMS / iClock push protocol the *biometric terminal is the HTTP client*: it periodically calls these routes to fetch its configuration handshake, upload attendance transactions and operation logs, and pull queued commands. Odoo never opens a connection to the device.

Every route is declared with `type='http'`, `auth='none'`, `csrf=False` and `save_session=False`, because the firmware carries no Odoo session and understands only plain text. All database access is therefore performed through `sudo()`, and the device is authenticated by its own serial number plus an optional shared `auth_token`. Every response is emitted as `Content-Type: text/plain` by the private helper `_text(body, status=200)`.

5.1 Route Summary

Method	Path	Purpose	Success response
GET	/iclock/cdata	Configuration handshake	Option block (see 5.3)
POST	/iclock/cdata	Upload ATTLOG / OPERLOG	`OK: <n>` or `OK`
GET	/iclock/getrequest	Pull queued commands	`C:<id>:<text>` lines, or `OK`
POST	/iclock/devicecmd	Acknowledge command result	`OK`
GET	/iclock/ping	Health check	`OK`

5.2 Security Model

The four data routes (`/iclock/cdata`, `/iclock/getrequest`, `/iclock/devicecmd`) share the same two-stage gate before any work is done. `/iclock/ping` is deliberately exempt.

5.2.1 Device recognition (`_get_device`)

The controller reads the serial number from the `SN` query/form parameter and calls `zkteco.biometric.device._get_registered_device(sn)`, which searches by `serial_number`. A device is treated as **unknown** — and the request rejected — when either:

- no record matches the serial number, or
- the matched record is still in the `draft` state.

In other words, the terminal must correspond to a device that has been **Confirmed** (state `confirmed`, `online` or `offline` — anything except `draft`). Confirming a device via `action_confirm()` also auto-generates an `auth_token` (a `uuid4().hex`) if none was set.

5.2.2 Token check (`_check_auth`)

If the recognized device has a non-empty `auth_token`, the token must be presented on every request. The controller looks, in order, at:

1. the `Authorization` HTTP header (an optional case-insensitive `Bearer` prefix is stripped),
2. the `X-ZKTeco-Token` HTTP header, or
3. a `token` query/form parameter.

The first value found is trimmed and compared for exact equality with the stored token. A device whose `auth_token` is empty is accepted unconditionally (open enrollment).

5.2.3 Rejection response

Both failure modes answer with the literal ADMS body `ACCESS DENIED` at **HTTP 401**, and log a warning containing the serial number and the client remote address. An unregistered or draft serial is denied through `_deny(sn)`; a bad/missing token returns the same body directly.

```
HTTP/1.1 401 Unauthorized
Content-Type: text/plain

ACCESS DENIED
```

> **Two distinct warnings are logged.** An unknown/draft serial logs "*access denied for unregistered/draft SN=…*", while a recognized device presenting no valid token logs "*rejected request for SN=… (missing or invalid auth token).*" Both use the WARNING level and include the remote address.

> On every successful (authorized) call to `/iclock/cdata`, `/iclock/getrequest` and `/iclock/devicecmd`, the controller invokes `device.mark_online()`, which refreshes `last_seen` and promotes the device state to `online`. `/iclock/ping` does *not* touch the device.

5.3 GET / POST /iclock/cdata

This single route serves two roles depending on the HTTP verb. It is bound to both `GET` and `POST` and passes the security gate described in 5.2 before dispatching.

5.3.1 GET — Configuration handshake

Purpose: the device requests its runtime options. **Query params:** `SN` (required), plus the auth token if configured. There is no request body.

The response is built by `device._build_handshake_options()`. Lines are joined with CRLF (`\r\n`). `Delay` comes from the device `heartbeat_interval` (default 30), `TransInterval` from `transaction_interval` (default 1), and `TimeZone` is the device timezone's UTC offset expressed in *whole minutes* (computed by `_get_timezone_offset_minutes()`).

```
GET OPTION FROM: OFAC192360001
Stamp=9999
OpStamp=9999
ErrorDelay=30
Delay=30
TransTimes=00:00;14:05
TransInterval=1
TransFlag=1111000000
TimeZone=330
Realtime=1
Encrypt=0
```

`Realtime=1` instructs the terminal to push punches immediately, and `Encrypt=0` disables payload encryption. `TimeZone=330` in the example above is UTC+05:30.

5.3.2 POST — Data upload (ATTLOG / OPERLOG)

Purpose: the device uploads records. **Query params:** `SN` (required), `table` (the record type; defaults to `ATTLOG` and is upper-cased), plus the auth token. **Body:** newline-separated records in the table-specific format below.

Dispatch is by table name:

- **ATTLOG** — calls `zkteco.biometric.attendance.log.ingest_attlog(device, body)`, which parses, de-duplicates and stores the punches, then matches employees and runs the processing engine. Responds `OK: <n>` where *n* is the number of newly created log records.
- **OPERLOG** — calls `ingest_operlog(device, body)`, a minimal scanner that counts operation-log lines starting with (or containing) the `USER` token. Responds `OK: <n>` where *n* is that count. This handler never raises.
- **Any other table** (e.g. OPTIONS, BIODATA) — simply acknowledged with `OK` and otherwise ignored.

```
POST /iclock/cdata?SN=OFAC192360001&table=ATTLOG HTTP/1.1

1001 2026-07-07 08:59:12 0 1 0
1002 2026-07-07 09:01:44 0 15 0
1001 2026-07-07 17:32:05 1 1 0
```

```
HTTP/1.1 200 OK
Content-Type: text/plain

OK: 3
```

ATTLOG line format

Each line is **tab-separated** (a whitespace fallback is tolerated, in which case the date and time are treated as two tokens and re-joined).

The columns are:

```
pin    YYYY-MM-DD HH:MM:SS    status    verify    workcode ...
```

- **pin** — the enrollment ID reported by the device; stored as `biometric_user_id` and matched against `hr.employee.biometric_user_id`.
- **timestamp** — device-local time in `%Y-%m-%d %H:%M:%S`. It is validated on parse (a line without a readable timestamp is discarded) and converted to naive UTC using the device timezone before storage in `punch_datetime`.
- **status** — the attendance status code (see 5.3.3), mapped to `punch_type` and kept verbatim in `status_code`.
- **verify** — the verification method code (see 5.3.3), mapped to `verify_mode`.
- Any further columns (workcode, etc.) are ignored; the whole original line is retained in `raw_payload`.

> Duplicate punches from offline replays are suppressed both in-batch and at the database level. Uniqueness is enforced by the `punch_uniq` constraint on `(device_id, biometric_user_id, punch_datetime, status_code)`; each row is created inside its own savepoint so a colliding replay is skipped silently rather than aborting the whole upload. The `OK: <n>` count therefore reflects only genuinely new punches.

5.3.3 Status and verify code maps

These maps live in `biometric_attendance_log.py` as module-level dicts (also exposed as class attributes `STATUS_MAP` / `VERIFY_MAP`).

status code	punch_type	Meaning
0	in	Check In
1	out	Check Out
2	break_out	Break Out
3	break_in	Break In
4	ot_in	Overtime In
5	ot_out	Overtime Out
any other	unknown	Fallback for unrecognized codes

verify code	verify_mode	Meaning
0	password	Password / PIN
1	fingerprint	Fingerprint
2	card	RFID card
9	palm	Palm
15	face	Face
any other	other	Fallback for unrecognized codes

5.4 GET /iclock/getrequest

Purpose: the polling device fetches any queued commands. **Query params:** `SN` (required) plus the auth token. No body. After the security gate and `mark_online()`, the response is produced by `device.get_pending_commands_payload()`.

That method searches `zkteco.device.command` records for this device in state `pending`, ordered by `create_date asc, id asc`. Each command is rendered on its own line as `C:<id>:<command_text>` (the *id* is the Odoo command record id), the lines are joined with a single `\n`, and every returned command is immediately flagged `sent` via `mark_sent()` (recording `sent_date`). When the queue is empty, the literal `OK` is returned.

```
C:42:INFO
C:43:SET OPTION Realtime=1,TransInterval=1,Delay=30
```

When there is nothing queued:

```
OK
```

> Commands are enqueued through `device.queue_command(command_type, command_text)`. For example the `Upload Settings` button (`action_upload_settings()`) queues an `INFO` command and a `SET OPTION Realtime=1,TransInterval=<n>,Delay=<n>` command, which are delivered on the device's next poll of this route.

5.5 POST /iclock/devicecmd

Purpose: the device reports the result of a previously delivered command. **Query params:** `SN` (required) plus the auth token. **Body:** URL-encoded fields `ID`, `Return` and `CMD`.

The controller parses the body with `urllib.parse.parse_qs(..., keep_blank_values=True)` and reads each field from the parsed body, falling back to the query/form kwargs. It then:

- coerces `ID` to an integer and searches for a matching `zkteco.device.command` constrained to both that id *and* this device,
- if found, calls `command.mark_done(return_code=<Return>, response=<CMD>)`,
- if not found, logs an INFO line ("*devicecmd ack for unknown command ID=...*") and does nothing further.

Inside `mark_done`, the command is marked `done` on a success return code and `error` otherwise. Success is defined as a `Return` value in `('0', 0, None, '')`; anything else is an error. The method stamps `ack_date` and stores the raw `return_code` and the echoed command name into `response`.

Regardless of whether a matching command was found, the route always answers `OK`.

```
POST /iclock/devicecmd?SN=OFAC192360001 HTTP/1.1
Content-Type: application/x-www-form-urlencoded

ID=42&Return=0&CMD=INFO
```

```
HTTP/1.1 200 OK
Content-Type: text/plain

OK
```

5.6 GET /iclock/ping

Purpose: a lightweight health check. This route takes no meaningful parameters, performs *no* serial-number lookup, *no* token check and does *not* update device presence. It unconditionally returns `OK` at HTTP 200, and is useful for verifying that the Odoo endpoint is reachable from the terminal's network without needing a registered device.

```
GET /iclock/ping HTTP/1.1

HTTP/1.1 200 OK
Content-Type: text/plain

OK
```

> Because `/iclock/ping` is entirely unauthenticated, treat a successful ping only as proof of connectivity — never as proof that a device is registered, confirmed, or permitted to upload data. Ingestion still requires passing the full 5.2 security gate.

6. Backend Usage Guide

This chapter walks an administrator through day-to-day operation of the module from the Odoo back office: finding the menus, registering and confirming a device, wiring employees to their biometric enrollment IDs, choosing a pairing policy, watching devices from the live dashboard, and auditing or reprocessing the raw punch log. Every screen described here is a native Odoo view shipped by the module; no external tooling is required.

6.1 Where the menus live

All screens are grafted onto the standard `Attendances` application. There are two entry points:

1. **Operational dashboard** — `Attendances › Biometric Devices`. This top-level item opens the device kanban/list/form and is visible to any user in the *Attendance Officer* group. Use it for the day-to-day view of your fleet.
2. **Configuration hub** — `Attendances › Configuration › Biometric`. This submenu is restricted to the *Attendance Manager* group and gathers everything an administrator needs:
   - `Devices` — the same device screen as above.
   - `Attendance Policies` — define how punches are paired into attendances.
   - `Biometric Logs` — the raw, read-only punch audit trail.
   - `Reprocess Logs` — open the reprocessing wizard.

> Employee-side configuration (the `Biometric ID` and policy assignments) does not have its own menu — it lives on each employee record, described in section 6.6.

6.2 Registering a device

From `Attendances › Biometric Devices`, create a new record and fill in the header and the **Identity** / **Security** groups:

1. Give the device a human-readable `Device Name` (e.g. *Main Entrance SpeedFace*). This is the title of the form.
2. Enter the `Serial Number` exactly as printed on / shown by the terminal. This is the key the device uses to identify itself when it pushes data, so it must match the hardware character-for-character.
3. Optionally set `Location`, `Time Zone` (`tz`), `Firmware Version` and, in a multi-company database, the owning `Company`.
4. The **Security** group shows the `Auth Token` (masked as a password) and `Last Seen` timestamp. Leave the token blank at creation — it is generated automatically when you confirm the device (see 6.3).

Network parameters and verification methods are entered on the notebook tabs and are covered in section 6.5.

6.3 The device lifecycle: Draft → Confirmed → Online / Offline

A device moves through four states, shown in the status bar at the top of the form (`Draft › Confirmed › Online › Offline`). The header buttons that drive these transitions appear and disappear depending on the current state:

Button (label)	Method	Shown when	What it does
Confirm	`action_confirm`	state = Draft	Validates the record and generates the `Auth Token` the terminal will use to authenticate its pushes. Moves the device to **Confirmed**.
Push Settings	`action_upload_settings`	state ≠ Draft	Queues the current network / verification configuration to be delivered to the terminal on its next poll. (The task brief refers to this as "Upload Settings"; the on-screen label is **Push Settings**.)
Mark Offline	`action_set_offline`	state = Confirmed or Online	Manually forces the device into the **Offline** state (e.g. when decommissioning or troubleshooting). Note the on-screen label is **Mark Offline**.
Reset to Draft	`action_reset_to_draft`	state ≠ Draft	Returns the device to **Draft** so identity/serial can be re-edited before re-confirming.

> Recommended flow: create the record → **Confirm** (token is minted) → configure the physical terminal to point at this Odoo server using the **Setup Guidance** tab → once the terminal handshakes, the device flips itself to **Online** and `Last Seen` starts updating. The transition to **Online**/**Offline** is driven by device traffic, not by an admin button.

6.4 Smart buttons: Logs and Commands

The button box at the top-right of the device form carries two statistic buttons:

- `Logs` (`action_view_logs`) — shows the `Log Count` and opens the biometric punch logs recorded by this device.
- `Commands` (`action_view_commands`) — shows the `Command Count` and opens the command queue for this device (reboot, sync time, set options, etc.).

6.5 The device form tabs

The notebook below the identity fields has five tabs:

Network

Two groups. **Addressing**: `IP Address` , `MAC Address` , `Subnet Mask` , `Gateway` , `Port` . **Push Timing**: `Transaction Interval` and `Heartbeat Interval` (how often the terminal uploads transactions and pings the server).

Verify Modes

Under **Enabled Verification Methods** are three toggles — `Verify Mode Face` , `Verify Mode Fingerprint` and `Verify Mode Palm` . A read-only `Verify Modes` summary field (`verify_modes_display`) shows the resulting human-readable list, which is also surfaced as a column in the device list view.

Setup Guidance

An instructional panel (not editable data) that tells you how to point the physical terminal at this server. For a SpeedFace-V5L it walks through `Menu › Comm. › Cloud Server Setting (ADMS)` : set **Server Mode** to **ADMS**, enter this server's address/domain, set **Server Port** to **8069** (HTTP) or **443** (HTTPS), and configure Ethernet (DHCP or static). The panel also echoes this device's `Serial Number` , `Port` and current `state` so you can cross-check them against the terminal. Once saved and rebooted, the terminal handshakes on `/iclock/cdata` and appears **Online** here.

Commands

A read-only list (no manual create) of queued `Device Commands` for this device, showing `Queued` date, `Command Type` , `Command Text` , delivery `State` (pending / sent / done / error, colour-coded), and the `Sent` , `Ack` and `Return Code` results. Commands are delivered the next time the terminal polls the queue.

Logs

A read-only list of this device's punches: `Punch Datetime` , `Biometric User ID` , `Employee` , `Punch Type` , `Verify Mode` and the `Process State` badge.

A chatter (message log / activities) is attached at the bottom of the form for audit notes.

6.6 Mapping employees and assigning policies

A punch is only converted into an attendance once the module can tie the device's enrollment number to an Odoo employee. Do this on each employee record, under `Employees › (open an employee) › HR Settings` tab, in the **Biometric Attendance (ZKTeco)** group (visible to Attendance Officers):

1. `Biometric ID` (`biometric_user_id`) — the enrollment ID / PIN the person is registered under on the terminal. **This is the single most important mapping field.** Until it is set, that employee's punches land as *unmatched*.
2. `Biometric Policy` (`biometric_policy_id`) — the default policy used to turn this employee's punches into check-in / check-out records.
3. `Policy Assignments` (the inline list) — optional time-bounded overrides. Each line has `Date From` , `Date To` and a `Policy` , letting you apply different pairing rules over different date ranges (e.g. a temporary shift change).

The employee form also carries a `Biometric Logs` smart button so you can jump straight to that person's punch history.

Defining the policies

Policies themselves are created under `Configuration › Biometric › Attendance Policies` . Each policy has a `Name` , a **Pairing Mode** and a set of tuning parameters:

Mode (value)	Meaning
Device Status Code (`device_status`)	Default. Trusts the terminal's own check-in / check-out status code on each punch to decide direction.

Mode (value)	Meaning
First Check Time (`first_check`)	Pairs by ordering: the first punch of a period opens an attendance, the next closes it, and so on.
Flexible Period (`flexible`)	Pairs within a rolling shift span; a punch beyond the span starts a new attendance.
Work Schedule (Calendar) (`calendar`)	Pairs punches against the employee's working-schedule calendar.

The parameter fields are:

- `Max Shift Span (h)` (`flexible_window_hours` , default 12.0) — only shown for the *Flexible Period* and *Work Schedule (Calendar)* modes; a punch beyond this span from the open check-in starts a new attendance.
- `Duplicate Window (s)` (`dedup_window_seconds` , default 60) — a repeated punch within this many seconds is ignored.
- `Min Gap After Check-out (min)` (`min_gap_minutes` , default 1) — a check-in occurring within this many minutes of the previous check-out is merged back into that attendance rather than starting a new one.

Policies can be archived; an **Archived** ribbon appears on archived records.

6.7 The live device dashboard (kanban)

`Attendances › Biometric Devices` opens on a dashboard-style kanban. Each card shows the device name, a colour-coded state pill (**Online** green, **Offline** red, **Confirmed** blue, **Draft** grey), the serial number, and — when set — the IP address, location, and `Last Seen` time (or "Never" if the device has not yet reported). Along the footer, a clickable `Logs` count opens that device's punches, and a **pending** badge appears when the device has queued commands still waiting to be delivered.

Switch to the list view for a tabular overview (name, serial, IP, location, verify modes, state, last seen, log count). The search bar offers quick filters for **Online / Offline / Confirmed / Draft** and group-by **Status**, **Company** or **Location**.

6.8 Reviewing raw logs and their process state

Every push from a terminal is stored as an immutable row in the `Biometric Logs` audit trail (`Configuration › Biometric › Biometric Logs`). The list is create- and edit-disabled — you can inspect but not hand-author punches. Open a row to see the full record, including the original `Raw Payload` the device sent, plus the resolved `Employee` , the linked `Attendance` (`hr_attendance_id`) if one was generated, and a `Process Note` explaining the outcome.

The `Process State` badge is the key column for triage. Its values are:

Process State	Colour	Meaning
`pending`	amber	Received but not yet run through the attendance engine.
`done`	green	Successfully turned into (or merged into) an attendance record.
`unmatched`	amber	No employee has the matching `Biometric ID` , so the punch could not be attributed. Fix by assigning the ID (6.6) and reprocessing.
`duplicate`	grey	Suppressed as a repeat within the policy's duplicate window.
`skipped`	grey	Intentionally not turned into an attendance by the policy logic.
`error`	red	Processing raised an error; see the `Process Note` .

The log form header also carries a **Reprocess** button (`action_reprocess`) for re-running the engine on that single record. Search filters make bulk triage easy: **Processed / Unprocessed**, **Unmatched Employee**, **Errors**, **Duplicates** and **Today**, plus group-by device, employee, process state, punch type or punch date.

6.9 The Reprocess wizard

When you fix a mapping (a newly assigned Biometric ID), change a policy, or need to rebuild attendances after a data issue, use `Configuration › Biometric › Reprocess Logs` . The wizard lets you scope the work and previews how many rows will be affected before you commit.

1. **Restrict the selection.** Optionally choose one or more `Devices` (leave empty to include every device), and/or a `Date From` and `Date To` to bound the punch range.
2. **Choose a scope.** The `Matching Logs` counter updates live to show how many logs the current selection would touch.

3. **Run it** with the **Reprocess** button, or **Cancel**.

The three scopes behave differently:

Scope (value)	Which logs	Action taken	When to use
Only unmatched logs (`unmatched`) — default	Logs flagged `unmatched`, plus any still-unprocessed log that never resolved to an employee.	Re-runs employee matching and processing *without* resetting anything — it simply picks up newly assigned Biometric IDs.	Right after you set a missing `Biometric ID` on an employee (6.6). The safest, most common option.
All unprocessed logs (`unprocessed`)	Every log with `Processed` = false.	Resets those logs to `pending`/unprocessed, then re-matches and re-runs the engine.	When the engine was interrupted or a batch of punches never got processed and you want to push them all through.
Full rebuild (`all`)	*Every* log in the selected devices/date range, regardless of state.	**Deletes every attendance previously generated from the selected logs** (via their `hr_attendance_id` link), clears each log's processed flag, state, note and attendance link, then re-matches and rebuilds attendances from scratch.	After changing a policy or correcting bad historical data, when you want the generated attendances recreated cleanly.

> **Full rebuild is destructive but scoped.** Selecting **Full rebuild** shows an on-screen warning because it **unlinks** the attendances the module itself created from the selected logs before recreating them. Because it deletes-then-recreates from the source punches, the rebuild does not leave duplicate attendances. Only attendances generated by this module (those linked to a log) are removed — **manually entered attendances are never touched**. Always narrow the device/date selection first and check the `Matching Logs` count.

Whatever the scope, the wizard finishes by re-running employee matching and the processing engine on the selected logs, then shows a notification summarising how many logs were reprocessed and how many attendance records were affected, and reloads the web client so the results are immediately visible. If nothing matched the criteria, it instead shows a warning and closes without changes.

7. Security & Access Rights

Access control for the module is declared in two files that load in a fixed order: `security/security.xml` creates the groups, the settings privilege and the global multi-company record rules, and is loaded *before* `security/ir.model.access.csv` so that the groups referenced by the ACL rows already exist. This section describes exactly what those files grant.

7.1 Settings Privilege and Groups

Odoo 19 replaces the legacy `res.groups.category_id` mechanism with the `res.groups.privilege` model. The module defines a single privilege that both of its groups belong to, so they appear together under Human Resources on the user form:

XML ID	Model	Name	Sequence	Category
`res_groups_privilege_zkteco`	res.groups.privilege	ZKTeco Attendance	20	`base.module_category_human_resources`

Two dedicated groups are created, both attached to that privilege via `privilege_id`:

XML ID	Display name	Purpose
`group_zkteco_user`	ZKTeco Attendance / Read Only	Read-only access to biometric devices, raw punch logs, commands and attendance policies.
`group_zkteco_admin`	ZKTeco Attendance / Administrator	Full management of biometric devices, policies, commands and log reprocessing.

Group inheritance (implied groups)

The two module groups form a hierarchy through Odoo's `implied_ids` mechanism, and a third link ties the module into the standard HR Attendance security model:

- **Administrator implies Read-Only.** `group_zkteco_admin` declares `implied_ids = [(4, ref('group_zkteco_user'))]`, so every administrator automatically holds every read-only permission in addition to the write/create/unlink rights granted to the admin group.
- **HR Attendance Administrator implies ZKTeco Administrator.** The module extends the standard group `hr_attendance.group_hr_attendance_manager` by adding `group_zkteco_admin` to *its* `implied_ids`. The direction matters: anyone who is an HR Attendance Administrator therefore inherits full ZKTeco admin access out of the box, without being assigned the ZKTeco group explicitly.
- **Default admin user.** `base.user_admin` is granted `group_zkteco_admin` directly, so the built-in administrator can manage devices immediately after installation.

> The effective privilege chain is `HR Attendance Administrator` → `ZKTeco Administrator` → `ZKTeco Read-Only`. Granting the topmost group cascades all lower permissions automatically.

7.2 Model Access Rights (ir.model.access.csv)

The CSV grants CRUD permissions per model and group. The ZKTeco Administrator group has full read/write/create/unlink on all six models (including the transient reprocess wizard). The ZKTeco Read-Only group has read-only access to the five persistent models. In addition, the standard `hr_attendance.group_hr_attendance_officer` group is given read-only access to three models so HR Officers can consult device state, punch logs and policies without editing them.

Model	Administrator (group_zkteco_admin)	Read-Only (group_zkteco_user)	HR Officer (group_hr_attendance_officer)
`zkteco.biometric.device`	read / write / create / unlink	read	read
`zkteco.biometric.attendance.log`	read / write / create / unlink	read	read
`zkteco.device.command`	read / write / create / unlink	read	—
`hr.attendance.policy`	read / write / create / unlink	read	read
`zkteco.employee.policy.line`	read / write / create / unlink	read	—
`zkteco.attendance.reprocess.wizard`	read / write / create / unlink	—	—

What HR Officers can see. An HR Officer (not an Administrator) receives read access to biometric devices, raw punch logs and attendance policies only. They do *not* get access to device commands or per-employee policy assignment lines, and they cannot open the reprocess wizard.

The reprocess wizard (`zkteco.attendance.reprocess.wizard`) is exposed to the Administrator group alone. Read-Only users and HR Officers have no ACL row for it and cannot launch a log reprocess. Because it is a transient model it also has no record rule (see below).

7.3 Multi-Company Record Rules

Five **global** record rules (`ir.rule` with `global="True"`) enforce company isolation. Being global, they apply to *every* user regardless of group, and they are combined with the ACLs above using AND. Each rule uses the same domain, which admits records that belong to one of the user's allowed companies or that are company-agnostic:

```
['|', ('company_id', '=', False), ('company_id', 'in', company_ids)]
```

The leading `('company_id', '=', False)` branch deliberately keeps shared, company-less records (for example a device or policy left unassigned to any company) visible to all users. The rules cover the five persistent models:

Rule XML ID	Model	Rule name
`rule_zkteco_biometric_device_company`	zkteco.biometric.device	ZKTeco Biometric Device: multi-company
`rule_zkteco_biometric_attendance_log_company`	zkteco.biometric.attendance.log	ZKTeco Attendance Log: multi-company
`rule_zkteco_device_command_company`	zkteco.device.command	ZKTeco Device Command: multi-company
`rule_hr_attendance_policy_company`	hr.attendance.policy	Attendance Policy: multi-company
`rule_zkteco_employee_policy_line_company`	zkteco.employee.policy.line	Employee Policy Assignment: multi-company

Because `company_ids` is the set of companies currently active in the user's company switcher, a user who has several companies enabled will see records from all of them at once. To view a single company's devices or logs in isolation, deselect the others in the top-right company selector.

The transient reprocess wizard has no record rule; company scoping for a reprocess run comes instead from the underlying logs and devices it operates on, which are already filtered by the rules above.

8. Device-Side Setup (SpeedFace-V5L) & Cron

This section covers the last mile: configuring the physical SpeedFace-V5L terminal so it pushes attendance to this Odoo server over the ADMS / iClock protocol, the in-app `Setup Guidance` tab that mirrors those steps, and the scheduled action that automatically retires devices that stop reporting.

8.1 Configuring the Terminal (SpeedFace-V5L)

All configuration is done on the terminal keypad. The goal is to register this Odoo instance as the device's ADMS (cloud) server so every punch is pushed to the `/iclock/cdata` endpoint. Work through the checklist below on the physical device.

1. Open `Menu > Comm. > Cloud Server Setting (ADMS)`.
2. Set `Server Mode` to **ADMS** (on some firmware this option is labelled **Domain Name**).
3. Enter the `Server Address / Domain Name` — this Odoo server's public IP address or domain name (the host the device must be able to reach).
4. Set `Server Port` to **8069** for plain HTTP, or **443** for HTTPS.
5. Enable `Enable Domain Name / HTTPS` when you use port **443**. A valid TLS certificate is required on the Odoo side for the handshake to succeed.
6. Under `Menu > Comm. > Ethernet`, choose **DHCP**, or set a **Static IP** together with a subnet mask and gateway that can route to the Odoo server.
7. Save and reboot the terminal. On restart the device performs its handshake on `/iclock/cdata` and, once matched to a confirmed record in Odoo, appears as **Online**.

> **Port and TLS must agree.** Port **443** requires HTTPS to be enabled on the terminal *and* a valid certificate served by Odoo (typically via a reverse proxy). Port **8069** is plain HTTP and needs HTTPS turned off on the device. A mismatch (HTTPS on with port 8069, or HTTPS off with port 443) is the most common reason a device never handshakes.

8.2 The In-App Setup Guidance Tab

Open a device record at `Attendance > Biometric Devices`, then the `Setup Guidance` notebook page on the form. This tab restates the same terminal checklist inside Odoo so an installer standing at the device does not need a separate document. It carries the heading "*Configure the device to push to Odoo (SpeedFace-V5L)*" and lists the `Menu > Comm. > Cloud Server Setting (ADMS)` steps described above.

Below the checklist, the tab echoes three read-only values from the current record so the on-device settings can be cross-checked against what Odoo expects:

Echoed field	Cross-check against the terminal
`Serial Number` (`serial_number`)	Must match the terminal's serial **exactly** — this is how Odoo identifies inbound push requests.
`Server Port` (`port`)	Should be the same port (8069 or 443) entered in the device's Cloud Server settings.
`Current registration state` (`state`)	Confirms whether the record is `draft`, `confirmed`, `online` or `offline` at a glance.

> Use the echoed **state** as your progress indicator. It should move `draft` → `confirmed` (after you press `Confirm` in Odoo) → `online` (once the rebooted terminal completes its first handshake). If it never leaves `confirmed`, the device is not reaching the server — recheck the server address, port and network settings.

8.3 The "Mark Stale Devices Offline" Cron

A device pushes periodic heartbeats while it is connected. If it loses power or network, no error is raised — it simply goes silent. The scheduled action `ZKTeco: Mark Stale Devices Offline` (XML id `cron_zkteco_mark_offline`) detects this and flips such devices to `offline` so the dashboard reflects reality.

Cron property	Value
Name	ZKTeco: Mark Stale Devices Offline
Model	`zkteco.biometric.device`
Executed code	`model._cron_mark_offline()`

Cron property	Value
Run as	`base.user_root` (OdooBot)
Interval	Every **5 minutes**
Priority	10
Active	Enabled by default

What each run does

The `_cron_mark_offline()` method performs the following on every run:

1. Searches all devices currently in state `online`.
2. For each one, reads the per-company threshold `zkteco_device_offline_minutes` (the `Device Offline After (min)` setting, default **5** minutes), and computes a cutoff of `now - threshold`.
3. Marks the device **stale** if it has no `last_seen` timestamp, or if `last_seen` is at or before the cutoff.
4. Writes `state = 'offline'` on all stale devices in one batch (if none are stale, it returns without changes).
5. Schedules a manager to-do activity for each newly offline device (see below).

The to-do activity

For every device that goes offline, the cron opens a `mail.mail_activity_data_todo` activity titled "*Biometric device offline*". The note names the device and serial and states it "has not reported since <last seen> (UTC) and was automatically marked offline", prompting the assignee to verify network connectivity and power. The assignee is chosen in this order of preference:

- the device's creator (`create_uid`), if that user is still active;
- otherwise the first active member of the `Attendance / Manager` group (`hr_attendance.group_hr_attendance_manager`);
- otherwise the current (cron) user as a last resort.

> The threshold is read from the device's **own company**, so multi-company deployments can tune tolerance independently under `Settings > Attendance`. Because the cron fires every 5 minutes but the default threshold is also 5 minutes, expect up to roughly 5–10 minutes of lag between a device going silent and it showing as `offline`. Raise `Device Offline After (min)` if a device with a long heartbeat interval is being flagged too eagerly.

8.4 Common Issues

> **Device stays in Draft.** A freshly registered device sits in `draft` until you press the `Confirm` button on the device form. Until it is confirmed, Odoo has not generated its authentication token and will not accept its pushes. Confirm the record, then reboot the terminal.
>
> **Terminal reports "ACCESS DENIED".** The server rejected the handshake. Either the serial number is not registered / not confirmed in Odoo (check that the terminal's serial matches the `Serial Number` echoed on the Setup Guidance tab exactly), or the auth token the device is sending does not match the one stored on the record. Reconfirm the device to be sure the token is in sync.
>
> **Punches show as "Unmatched".** The push arrived but Odoo could not tie the biometric user to an employee. Open the employee, set their `Biometric ID` (`biometric_user_id`) to the ID enrolled on the terminal, then run `Attendance > Reprocess Logs` to re-match and reprocess the affected logs without data loss.

9. Testing & Acceptance Criteria

The module ships an automated test suite at `tests/test_adms_protocol.py` that exercises the full ingestion and attendance engine: ATTLOG line parsing, the `ingest_attlog` pipeline, employee matching, every attendance policy mode, duplicate suppression, the offline-detection cron, and the HTTP controller's serial-number gate. The tests run against a deterministic fixture so timestamps never drift across a day boundary regardless of the host or company timezone.

9.1 Test Fixture & Isolation

All tests are tagged `@tagged('post_install', '-at_install')` so they execute after the module is fully installed. The unit tests use Odoo's `TransactionCase` (each test runs in its own transaction that is rolled back afterwards); the controller test uses `HttpCase` to issue real HTTP requests. Every fixture is rebuilt in `setUp()`, giving each test a clean, isolated, fully deterministic starting state.

The shared fixture built in `setUp()` consists of:

- A `resource.calendar` pinned to `tz='UTC'`, so `employee._get_tz()` resolves to `'UTC'` and punch timestamps are stable.
- A confirmed `zkteco.biometric.device` ("Test SpeedFace-V5L", serial `TESTSN0001`, `tz='UTC'`), promoted via `action_confirm()`.
- Two `hr.attendance.policy` records: one in `device_status` mode and one in `flexible` mode (`flexible_window_hours=12.0`), both with `dedup_window_seconds=60` and `min_gap_minutes=0`.
- One `hr.employee` ("Alice Tester") enrolled with `biometric_user_id='1001'` and pinned to the device-status policy via `biometric_policy_id`.

Three private helpers keep the tests concise: `_line()` builds a single tab-separated ATTLOG line; `_make_log()` creates a raw log directly (bypassing ingest) for engine-only unit tests; and `_employee_attendances()` fetches the employee's `hr.attendance` rows ordered by `check_in`.

9.2 Test Classes & Methods

Class `TestADMSProtocol` (TransactionCase)

Test method	What it asserts
`test_parse_attlog_line_valid`	A well-formed tab-separated ATTLOG line parses into a dict with the correct `biometric_user_id`, `datetime_str`, `status_code`, `verify_code`, and a `raw` key.
`test_parse_attlog_line_whitespace_fallback`	A line with spaces instead of tabs still parses; the parser rebuilds the timestamp from two tokens and recovers PIN, datetime, and status.
`test_parse_attlog_line_blank`	Blank/empty input returns `None` for `''`, whitespace-only, tab-only, and `None`.
`test_parse_attlog_line_malformed`	Malformed input returns `None`: a single garbage token, a lone PIN, an unparseable datetime column, and an empty PIN.
`test_ingest_attlog_creates_logs`	`ingest_attlog` creates exactly one log; the timestamp is stored as naive UTC, `punch_type='in'`, `verify_mode='fingerprint'`, the employee is matched, and the log is `processed` with `process_state='done'` and a linked `hr_attendance_id`.
`test_ingest_attlog_dedups_identical_replay`	Re-ingesting the identical line creates nothing (count 0, empty recordset) thanks to the `punch_uniq` DB constraint; the total log count is stable across the replay.
`test_match_employee_links_employee`	`_match_employee()` links an unassigned log to the employee whose `biometric_user_id` matches the punch PIN.
`test_match_employee_unmatched`	A punch for an unknown PIN (`9999`) leaves `employee_id` empty and sets `process_state='unmatched'`.
`test_device_status_creates_then_closes_attendance`	Under `device_status`, a status-0 punch opens an attendance (check_in set, no check_out); a later status-1 punch closes the *same* attendance (check_out set, `punch_type='out'`) without creating a second record.

Test method	What it asserts
`test_first_check_policy`	Under `first_check`, three punches in a day collapse to one attendance: the first punch is check_in, the latest is check_out; all logs are marked processed.
`test_flexible_policy_toggle_within_window`	Under `flexible`, a second punch inside the 12h window closes the open attendance (one record, check_in/check_out set correctly).
`test_flexible_policy_beyond_window_starts_new`	Under `flexible`, two punches 14.5h apart exceed the 12h window: the first session is treated as a forgotten check-out and a new attendance is opened — two records, the latest one still open.
`test_dedup_within_window`	Two 'in' punches 30s apart (within the 60s dedup window) both persist as logs, but the second is flagged `process_state='duplicate'` and only one attendance is opened by the first punch.
`test_cron_mark_offline_flips_stale_device`	`_cron_mark_offline()` flips an online device to `state='offline'` when `last_seen` is older than the company's `zkteco_device_offline_minutes` threshold.
`test_cron_mark_offline_keeps_fresh_device`	`_cron_mark_offline()` leaves a device seen "just now" as `state='online'` — a fresh device is never flipped offline.

Class `TestADMSController` (HttpCase)

Test method	What it asserts
`test_cdata_unregistered_sn_denied`	A GET to `/iclock/cdata?SN=doesnotexist` from an unregistered serial number is rejected with HTTP `401` and a body containing `ACCESS DENIED`.

9.3 Result Achieved

16 tests, all passing. The module installs clean on a fresh database with **0 errors and 0 warnings**. The suite covers parsing (valid / whitespace-fallback / blank / malformed), ingestion and identical-replay de-duplication, employee matching (linked and unmatched), all four attendance behaviours (`device_status` open-then-close, `first_check`, `flexible` within/beyond window), in-window duplicate suppression, the offline-detection cron (stale-flip and fresh-keep), and the controller's unregistered-serial gate.

9.4 Running the Tests

Run the full suite against a database with the test tags enabled and exit immediately after:

```
python odoo-bin -c odoo.conf -d <db> -u zkteco_adms_attendance --test-enable --stop-after-init
```

`--test-enable` activates the test runner, `-u zkteco_adms_attendance` upgrades (and re-loads) the module so its tests are collected, and `--stop-after-init` shuts the server down as soon as the run finishes. Because the tests are tagged `post_install`, they execute against the fully installed module. A clean run reports 16 tests passed with no errors or warnings.

Replace `<db>` with your target database name. Run against a dedicated test database — the suite is transactional and rolls back its own data, but running `-u` upgrades the module in whatever database you name.